
DRL-Project

MSE
FS 2026

Reader

EVA Deep Reinforcement Learning im Modul CE | WÜRC

1 Deep Reinforcement Learning Project (EVA)

In order to become industry-ready and thrive in today's world, it is essential that we know 3R's (reading, writing & arithmetic) and 4C's (creativity, critical thinking, communication, collaboration) that can be very effective in making you stand out of the crowd. Therefore the focus is increasing towards project-based learning more than traditional instruction. It is believed that making projects gives you the opportunity to ask questions and apply your knowledge towards solving a problem. As the job market is becoming more competitive, it is imperative that you continue upskilling yourself and have proof to show the employers. Without much ado here is a list of reinforcement learning project ideas from beginner to advanced level.

Within the EVA Deep Reinforcement Learning, you have to hand in at the end of the semester a practical application of RL in form of a working Python code. Each WR project must be submitted to the instructor in a timely manner in the form of an executable Jupyter notebook or Python code (Python ≥ 3.6) with appropriate documentation. The Jupyter notebook or Python code documentation should include the following sections:

1.1 Sections of the Report

- (a) *header*: Project name, author, date, OST logo.
- (b) *Introduction*: Description of the task and scope.
- (c) *Objective and approach*: What is to be achieved? What approach do you take to achieve this goal?

- (d) *Description of the environment and the agent*: What is the state $\mathcal{S}$ and the action space $\mathcal{A}$.
- (e) *Description of the algorithm*: What algorithm is used? Is it a policy optimization or Q-learning approach? In case of Deep Reinforcement Learning, what model is used for the prediction?
- (f) *Train and Test*: Show learning curves for different parametersets (α, γ) and evaluate the performance of the agent after the training statistically.
- (g) *Conclusion and Outlook*: What could be investigated further? What other methods would be suitable? Are there improvements to the model or type of agent?
- (h) *References*:
References of relevant papers, books, internet links of sources and data used.

1.2 Requirements

The following **requirements** apply to the project:

- (a) The DRL-project is done *individually*.
- (b) If you use ideas or code from other sources, those sources must be clearly identified. Include a *declaration of independence* with your Jupyter notebook or documentation (see at the end of the document).
- (c) Your DRL project must be about new things you have done this semester (HS2022). You may not use older work that you have already performed.
- (d) Timely submission of the documentation, the Python code or the jupyter notebook of the DRL project is required for participation in the exam interview and to pass the EVA.

If you have trouble writing a proposal, feel free to contact the instructor. Once you have submitted your proposal, we will give you feedback. Of course, the final responsibility for defining and executing an interesting topic rests with you.

2 Dates and evaluation of the project

2.1 Dates

project goal	date	scope
Proposal	KW10	1 page
Final Jupyter notebook or Python code	17.6.2026 (KW25)	-

2.2 Evaluation of the Project

Each outcome of your project will be evaluated based on the following factors:

- (a) *Completeness*: Is the documentation complete? Is the code executable and complete including a description of the dependencies, package requirements, declaration of independence?
- (b) *Comprehensibility*: Is the code and description comprehensible? Judgement of the writing style and clarity of the written work (report, inline documentation in the code).
- (c) *Correctness*: Were the methods applied correctly? Was the theory behind the agent really understood?
- (d) *Scope* What is the scope and size of the study, of training, tests, parameter studies and experiments?
- (e) *Quality*: Quality and style of the code (cf. Zen of Python 5), performance of the agent, quality and style of visualization and graphs.

3 Project Proposals

The following links could give you some ideas, what can be achieved within this DRL projects.

- (a) **RLlib**: Industry-Grade Reinforcement Learning
- (b) Solve toy problems with **OpenAI Gym** (beginner-friendly)
- (c) Play **Atari games** from pixel input with OpenAI Gym
- (d) Simulate control tasks with **PyBullet**
- (e) Create your own reinforcement learning environment with **Unity ML-Agents**
- (f) Race self-driving cars with **AWS DeepRacer** (beginner-friendly)
- (g) Mine diamonds in Minecraft with **MineRL**
- (h) Join the community at **AIArena** building agents for StarCraft II
- (i) Build a Chess Bot with **OpenSpiel**
- (j) Predict stock prices with **TensorTrade**
- (k) Train cooperative agents with **PettingZoo**
- (l) Build a Poker bot with **RLCard**
- (m) Join an AI Programming competition
- (n) Training Humanoids for Control Tasks using **MuJoCo**
- (o) Play **Super Mario Bros** with a Double Deep Q-Network

4 Declaration of Independence

You should sign the following self-declaration and attach it to the submitted Jupyter Notebook, your documentation and Python code.

«I hereby certify that I have written this thesis independently and have not used any auxiliary materials other than those indicated. The passages of the work, which are taken in the wording or the sense after other works (to it also Internet sources count), were marked under indication of the source ».

5 Zen of Python

```
1  """
2  Beautiful is better than ugly.
3  Explicit is better than implicit.
4  Simple is better than complex.
5  Complex is better than complicated.
6  Flat is better than nested.
7  Sparse is better than dense.
8  Readability counts.
9  Special cases aren't special enough to break the rules.
10 Although practicality beats purity.
11 Errors should never pass silently.
12 Unless explicitly silenced.
13 In the face of ambiguity, refuse the temptation to guess.
14 There should be one - and preferably only one - obvious way to do it.
15 Although that way may not be obvious at first unless you're Dutch.
16 Now is better than never.
17 Although never is often better than right now.
18 If the implementation is hard to explain, it's a bad idea.
19 If the implementation is easy to explain, it may be a good idea.
20 Namespaces are one honking great idea -- let's do more of those!
21 """
```